

Guía de Consulta — Arquitectura Models / Routes / Controllers + Populate

Importante: este ejemplo usa un dominio distinto (una plataforma de música: Artistas, Canciones, Playlists) para que entiendas el patrón sin copiar directamente las colecciones de su proyecto de Numerología. Ustedes deben replicar esta misma estructura con Usuarios, MatricesNumerologicas, PromptsConfig, LecturasIA y BitacoraTransacciones.

1. Estructura de carpetas que deben crear

```
proyecto-numerologia/
├── src/
│   ├── config/
│   │   └── db.js
│   ├── models/
│   │   ├── Artista.model.js
│   │   ├── Cancion.model.js
│   │   └── Playlist.model.js
│   ├── controllers/
│   │   ├── artista.controller.js
│   │   ├── cancion.controller.js
│   │   └── playlist.controller.js
│   └── routes/
│       ├── artista.routes.js
│       ├── cancion.routes.js
│       └── playlist.routes.js
├── .env
├── .gitignore
├── package.json
└── index.js
```

Responsabilidad de cada carpeta:

Carpeta	Responsabilidad	Lo que NO debe hacer
models/	Define el esquema (schema) y las validaciones de Mongoose	No debe tener lógica de negocio ni req/res
controllers/	Contiene la lógica: recibe req, consulta el modelo, responde con res	No debe definir rutas ni schemas
routes/	Solo mapea método HTTP + path → función del controller	No debe tener lógica, solo router.metodo (path, controller)
config/	Conexión a la base de datos, variables de entorno centralizadas	No debe mezclarse con modelos

2. package.json — habilitar ES Modules

```
{
  "name": "proyecto-numerologia",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "node --watch index.js"
  },
  "dependencies": {
    "express": "^4.19.2",
    "mongoose": "^8.5.0",
    "dotenv": "^16.4.5"
  }
}
```

Con "type": "module" ya pueden usar import/export en todos los archivos .js sin necesidad de extensión .mjs.

3. .env

```
PORT=3000
MONGO_URI=mongodb://lo.....
```

4. src/config/db.js

```
// db.js — centraliza la conexión para no repetir mongoose.connect en cada
archivo
import mongoose from "mongoose";

export const conectarDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB conectado correctamente");
  } catch (error) {
    // si la conexión falla, no tiene sentido seguir levantando el servidor
    console.error("Error al conectar a MongoDB:", error.message);
    process.exit(1);
  }
};
```

5. Modelos (Models)

Usamos 3 colecciones relacionadas para que quede clarísimo cuándo usar ref simple, ref en array, y populate anidado.

src/models/Artista.model.js

```
import mongoose from "mongoose";

const artistaSchema = new mongoose.Schema({
  nombre_artistico: {
    type: String,
    required: true,
    trim: true,
  },
  pais: {
```

```

    type: String,
    required: true,
  },
  genero_musical: {
    type: String,
    enum: ["Reggaeton", "Salsa", "Pop", "Rock", "Vallenato"],
    default: "Pop",
  },
  verificado: {
    type: Boolean,
    default: false,
  },
  fecha_debut: {
    type: Date,
    required: true,
  },
}, { timestamps: true }); // agrega createdAt / updatedAt automáticamente

export default mongoose.model("Artista", artistaSchema);

```

src/models/Cancion.model.js

```

import mongoose from "mongoose";

const cancionSchema = new mongoose.Schema({
  titulo: {
    type: String,
    required: true,
  },
  // Relación 1 a muchos: una canción pertenece a UN artista
  artista_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Artista", // debe coincidir EXACTO con el nombre usado en
mongoose.model()
    required: true,
  },
  duracion_segundos: {
    type: Number,
    required: true,
  },
  reproducciones: {
    type: Number,
    default: 0,
  },
  fecha_lanzamiento: {
    type: Date,
    default: Date.now,
  },
}, { timestamps: true });

export default mongoose.model("Cancion", cancionSchema);

```

src/models/Playlist.model.js

```

import mongoose from "mongoose";

const playlistSchema = new mongoose.Schema({
  nombre: {
    type: String,
    required: true,
  },
  creador: {
    type: String,

```

```

    required: true,
  },
  // Relación muchos a muchos: una playlist tiene VARIAS canciones
  canciones: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: "Cancion",
  }],
  publica: {
    type: Boolean,
    default: true,
  },
}, { timestamps: true });

export default mongoose.model("Playlist", playlistSchema);

```

6. Controllers — CRUD + los 3 tipos de populate

src/controllers/artista.controller.js

```

import Artista from "../models/Artista.model.js";

// Crear artista
export const crearArtista = async (req, res) => {
  try {
    const { nombre_artistico, pais, genero_musical, fecha_debut } = req.body;
    const artista = await Artista.create({ nombre_artistico, pais,
    genero_musical, fecha_debut });
    // "verificado" NO se destructura: ese campo lo debería setear un admin en
    otro endpoint, no el registro público
    res.status(201).json(artista);
  } catch (error) {
    res.status(400).json({ mensaje: "Error al crear artista", error:
    error.message });
  }
};

// Actualizar artista
export const actualizarArtista = async (req, res) => {
  try {
    const { nombre_artistico, pais, genero_musical } = req.body;
    const artista = await Artista.findByIdAndUpdate(
      req.params.id,
      { nombre_artistico, pais, genero_musical },
      { new: true, runValidators: true }
    );
    if (!artista) return res.status(404).json({ mensaje: "Artista no
    encontrado" });
    res.status(200).json(artista);
  } catch (error) {
    res.status(400).json({ mensaje: "Error al actualizar artista", error:
    error.message });
  }
};

// Eliminar artista
export const eliminarArtista = async (req, res) => {
  try {
    const artista = await Artista.findByIdAndDelete(req.params.id);
    if (!artista) return res.status(404).json({ mensaje: "Artista no
    encontrado" });
    res.status(200).json({ mensaje: "Artista eliminado correctamente" });
  }
};

```

```

    } catch (error) {
      res.status(500).json({ mensaje: "Error al eliminar artista", error:
error.message });
    }
  };

// Listar todos
export const listarArtistas = async (req, res) => {
  try {
    const artistas = await Artista.find();
    res.status(200).json(artistas);
  } catch (error) {
    res.status(500).json({ mensaje: "Error al listar artistas", error:
error.message });
  }
};

// Obtener uno por id
export const obtenerArtista = async (req, res) => {
  try {
    const artista = await Artista.findById(req.params.id);
    if (!artista) return res.status(404).json({ mensaje: "Artista no
encontrado" });
    res.status(200).json(artista);
  } catch (error) {
    res.status(500).json({ mensaje: "Error al obtener artista", error:
error.message });
  }
};

```

src/controllers/cancion.controller.js — populate simple

```

import Cancion from "../models/Cancion.model.js";

export const crearCancion = async (req, res) => {
  try {
    // NUNCA se pasa req.body directo a create(): ver sección 12 (Mass
Assignment).
    // Deestructuramos solo los campos que el cliente debe poder enviar.
    const { titulo, artista_id, duracion_segundos, fecha_lanzamiento } =
req.body;
    const cancion = await Cancion.create({ titulo, artista_id,
duracion_segundos, fecha_lanzamiento });
    // "reproducciones" queda fuera a propósito: nadie debería poder inflarlo
desde el body
    res.status(201).json(cancion);
  } catch (error) {
    res.status(400).json({ mensaje: "Error al crear canción", error:
error.message });
  }
};

export const actualizarCancion = async (req, res) => {
  try {
    const { titulo, duracion_segundos, fecha_lanzamiento } = req.body;
    const cancion = await Cancion.findByIdAndUpdate(
      req.params.id,
      { titulo, duracion_segundos, fecha_lanzamiento },
      { new: true, runValidators: true } // new: devuelve el doc actualizado;
runValidators: respeta el schema
    );
    if (!cancion) return res.status(404).json({ mensaje: "Canción no
encontrada" });
  }
};

```

```

    res.status(200).json(cancion);
  } catch (error) {
    res.status(400).json({ mensaje: "Error al actualizar canción", error:
error.message });
  }
};

export const eliminarCancion = async (req, res) => {
  try {
    const cancion = await Cancion.findByIdAndDelete(req.params.id);
    if (!cancion) return res.status(404).json({ mensaje: "Canción no
encontrada" });
    res.status(200).json({ mensaje: "Canción eliminada correctamente" });
  } catch (error) {
    res.status(500).json({ mensaje: "Error al eliminar canción", error:
error.message });
  }
};

// POPULATE SIMPLE: en lugar de devolver solo el artista_id (ObjectId),
// trae el documento completo del Artista relacionado.
export const listarCancionesConArtista = async (req, res) => {
  try {
    const canciones = await Cancion.find()
      .populate("artista_id", "nombre_artistico pais verificado");
    // el segundo argumento (string) selecciona SOLO esos campos del artista,
    // así evitas traer info innecesaria (buena práctica de rendimiento)
    res.status(200).json(canciones);
  } catch (error) {
    res.status(500).json({ mensaje: "Error al listar canciones", error:
error.message });
  }
};

```

src/controllers/playlist.controller.js — populate en array + populate anidado

```

import Playlist from "../models/Playlist.model.js";

export const crearPlaylist = async (req, res) => {
  try {
    const { nombre, creador, canciones, publica } = req.body;
    const playlist = await Playlist.create({ nombre, creador, canciones,
publica });
    res.status(201).json(playlist);
  } catch (error) {
    res.status(400).json({ mensaje: "Error al crear playlist", error:
error.message });
  }
};

export const actualizarPlaylist = async (req, res) => {
  try {
    const { nombre, canciones, publica } = req.body;
    const playlist = await Playlist.findByIdAndUpdate(
      req.params.id,
      { nombre, canciones, publica },
      { new: true, runValidators: true }
    );
    if (!playlist) return res.status(404).json({ mensaje: "Playlist no
encontrada" });
    res.status(200).json(playlist);
  } catch (error) {

```

```

        res.status(400).json({ mensaje: "Error al actualizar playlist", error:
error.message });
    }
};

export const eliminarPlaylist = async (req, res) => {
    try {
        const playlist = await Playlist.findByIdAndDelete(req.params.id);
        if (!playlist) return res.status(404).json({ mensaje: "Playlist no
encontrada" });
        res.status(200).json({ mensaje: "Playlist eliminada correctamente" });
    } catch (error) {
        res.status(500).json({ mensaje: "Error al eliminar playlist", error:
error.message });
    }
};

// POPULATE EN ARRAY: "canciones" es un array de ObjectId,
// Mongoose reemplaza cada elemento por el documento completo de Cancion.
export const obtenerPlaylistConCanciones = async (req, res) => {
    try {
        const playlist = await Playlist.findById(req.params.id)
            .populate("canciones");
        if (!playlist) return res.status(404).json({ mensaje: "Playlist no
encontrada" });
        res.status(200).json(playlist);
    } catch (error) {
        res.status(500).json({ mensaje: "Error al obtener playlist", error:
error.message });
    }
};

// POPULATE ANIDADO: trae las canciones de la playlist, Y DENTRO de cada canción
// trae también el artista relacionado (populate dentro de populate).
// Esto es EXACTAMENTE lo que van a necesitar cuando en su proyecto quieran,
// por ejemplo, traer una LecturaIA con su Usuario Y el PromptsConfig usado.
export const obtenerPlaylistCompleta = async (req, res) => {
    try {
        const playlist = await Playlist.findById(req.params.id)
            .populate({
                path: "canciones",
                select: "titulo duracion_segundos reproducciones",
                populate: {
                    path: "artista_id",
                    select: "nombre_artistico genero_musical",
                },
            });
        if (!playlist) return res.status(404).json({ mensaje: "Playlist no
encontrada" });
        res.status(200).json(playlist);
    } catch (error) {
        res.status(500).json({ mensaje: "Error al obtener playlist completa", error:
error.message });
    }
};

```

7. Routes — solo mapean, no tienen lógica

src/routes/artista.routes.js

```
import { Router } from "express";
```

```
import {
  crearArtista,
  listarArtistas,
  obtenerArtista,
  actualizarArtista,
  eliminarArtista,
} from "../controllers/artista.controller.js";

const router = Router();

router.post("/", crearArtista);
router.get("/", listarArtistas);
router.get("/:id", obtenerArtista);
router.put("/:id", actualizarArtista);
router.delete("/:id", eliminarArtista);

export default router;
```

src/routes/cancion.routes.js

```
import { Router } from "express";
import {
  crearCancion,
  listarCancionesConArtista,
  actualizarCancion,
  eliminarCancion,
} from "../controllers/cancion.controller.js";

const router = Router();

router.post("/", crearCancion);
router.get("/", listarCancionesConArtista);
router.put("/:id", actualizarCancion);
router.delete("/:id", eliminarCancion);

export default router;
```

src/routes/playlist.routes.js

```
import { Router } from "express";
import {
  crearPlaylist,
  obtenerPlaylistConCanciones,
  obtenerPlaylistCompleta,
  actualizarPlaylist,
  eliminarPlaylist,
} from "../controllers/playlist.controller.js";

const router = Router();

router.post("/", crearPlaylist);
router.get("/:id", obtenerPlaylistConCanciones);
router.get("/:id/completa", obtenerPlaylistCompleta);
router.put("/:id", actualizarPlaylist);
router.delete("/:id", eliminarPlaylist);

export default router;
```

8. Integración en index.js (punto de entrada único)

```
import "dotenv/config"; // carga las variables del .env ANTES de todo lo demás
```

```
import express from "express";
import { conectarDB } from "../src/config/db.js";
import artistaRoutes from "../src/routes/artista.routes.js";
import cancionRoutes from "../src/routes/cancion.routes.js";
import playlistRoutes from "../src/routes/playlist.routes.js";

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json()); // necesario para leer JSON en el body de los POST/PUT

// cada módulo de rutas se monta bajo su propio prefijo /api/v1/...
app.use("/api/v1/artistas", artistaRoutes);
app.use("/api/v1/canciones", cancionRoutes);
app.use("/api/v1/playlists", playlistRoutes);

// middleware de manejo de errores no capturados (opcional pero recomendado)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ mensaje: "Error interno del servidor" });
});

conectarDB().then(() => {
  app.listen(PORT, () => {
    console.log(`Servidor corriendo en http://localhost:${PORT}`);
  });
});
```

Aplicado a su proyecto: el `index.js` de Numerología debe verse igual, solo cambiando los `import` por sus rutas: `usuarios.routes.js`, `numerologia.routes.js`, `lecturas.routes.js`, etc., cada una montada bajo `/api/v1/...` como indica el enunciado.

10. req.body directo a create() — ¿mala práctica?

Mongoose por defecto usa `strict: true`, así que un campo que **no existe** en el schema simplemente se descarta al guardar — eso no es el riesgo. El riesgo real es **mass assignment**: cuando el cliente envía un campo que **sí existe** en el schema pero no debería poder decidirlo él.

Ejemplo con el modelo Artista:

```
// ❌ Mal: el cliente puede autoverificarse en el registro
const artista = await Artista.create(req.body);
// req.body = { nombre_artistico: "Cualquiera", verificado: true }
// → se guarda "verificado: true" sin que nadie lo haya aprobado

// ✅ Bien: solo entran los campos que el cliente debe controlar
const { nombre_artistico, pais, genero_musical, fecha_debut } = req.body;
const artista = await Artista.create({ nombre_artistico, pais, genero_musical,
fecha_debut });
// "verificado" se setea después, desde un endpoint de admin (PATCH
/artistas/:id/verificar)
```

Regla práctica: cualquier campo que represente un estado controlado por el sistema o por un rol superior (`verificado`, `reproducciones`, `activo`, `tokens_consumidos`, `roles/permisos`, `saldos`, etc.) **nunca** debe salir de un destructuring de `req.body` en el `create/update` público. Todos los ejemplos de esta guía (sección 6) ya siguen este patrón.

Aplicado a su proyecto: en `PromptsConfig`, el campo activo es exactamente este caso — no debería poder activarse/desactivarse desde el mismo endpoint donde se crea la plantilla.

11. Checklist para que repliquen en Numerología

- ☐ Crear las 4 carpetas (`config`, `models`, `controllers`, `routes`) + `index.js` en la raíz
- ☐ `package.json` con `"type": "module"`
- ☐ Un archivo de modelo por colección (5 en total: `Usuarios`, `MatricesNumerologicas`, `PromptsConfig`, `LecturasIA`, `BitacoraTransacciones`)
- ☐ Identificar qué campos son `ref` (ej. `MatricesNumerologicas.usuario_id` → `ref Usuario`; `LecturasIA.usuario_id` y `LecturasIA.prompt_usado_id` → dos refs distintos en el mismo documento)
- ☐ Al menos un endpoint con `populate` simple (ej. traer una Matriz con los datos del Usuario)
- ☐ Al menos un endpoint con `populate` anidado (ej. traer una `LecturaIA` con el Usuario y el `PromptsConfig` usado, a la vez)
- ☐ `index.js` montando cada router bajo su propio prefijo `/api/v1/...`
- ☐ Manejo de errores con `try/catch` en cada controller (nunca dejar una promesa sin capturar)
- ☐ CRUD completo (`create`, `read`, `update`, `delete`) en cada módulo — no solo `create/get`
- ☐ Destructuring explícito de `req.body` en `create/update`, nunca pasar el objeto completo (ver sección 11)